DEEP LEARNING

Deep Neural Networks Part I

Subhankar Roy

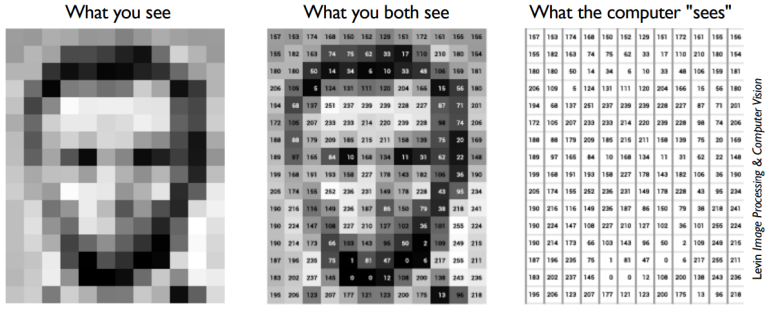
Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Table of Contents

1. **Fully Connected Neural Network**

2. **Convolutional Neural Network**
   - 2.1 Filters
   - 2.2 Feature Maps
   - 2.3 Padding
   - 2.4 Pooling
   - 2.5 Architecture
   - 2.6 Visualization

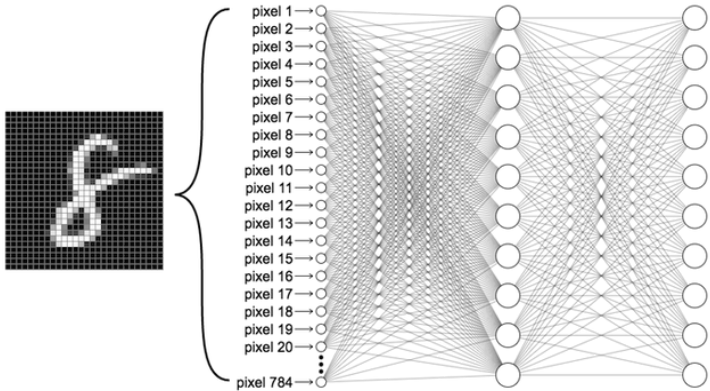
Today: From Pixels to Concepts



- A **pixel** is the smallest unit of a digital image. It represents a single point of color or intensity.
- An image is a collection of pixels. A grayscale image is nothing but a matrix of numbers that ranges between $[0, 255]$.
- Looking at pixels, how can a computer answer the question: "*Who is in the image?*"

Fully Connected Neural Network

MLP for Image Classification



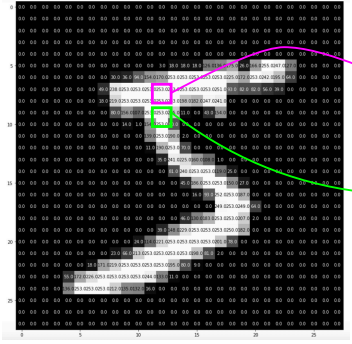
- When we have 2D image as input, we first flatten the image into a column vector.
- Then we treat this as an input feature and feed it to the MLP.
- MLPs are made of fully connected layers, where each input is connected to all the neurons. We often call MLPs by the name Fully Connected Neural Network (FCNN).

Why Not MLPs for Images?

- Limitation #1: High-Dimensional Input = Huge Number of Parameters.
  - For a small 100×100 grayscale input image we will require 10,000 input neurons in the input layer of the MLP.
  - If we have one hidden layer with 1,000 units, then we have a transformation matrix of size $10000 \times 1000 = 10$M parameters!
  - Quickly becomes computationally infeasible if we have a deeper network.
  - In real applications we deal with higher resolution images (e.g., 1K resolution is 1024 × 768, that is 786K pixels).

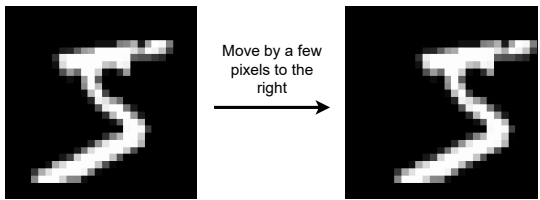
Why Not MLPs for Images?

- Limitation #2: No Awareness of Spatial Structure due to Flatenning.
  - MLP ignores the image's 2D structure and treats every pixel independently.
  - Doesn't know nearby pixels are related. For e.g., an edge at pixel (10, 10) and (12, 10) becomes two distant values in a vector.
  - Thus, it loses the idea of local patterns or shapes (e.g., edges).
  - It has to learn from scratch what local patterns are, i.e.wasting computation.



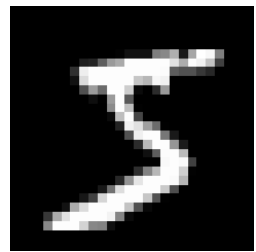
Why Not MLP for Images?

- Limitation #3: Poor Generalization.
  - Too many parameters → overfitting → poor generalization.
  - Small shifts in the image (e.g., moving a digit slightly) lead to completely different inputs to the MLP.
  - MLPs don't recognize that the object is the same, just shifted – no translation invariance.



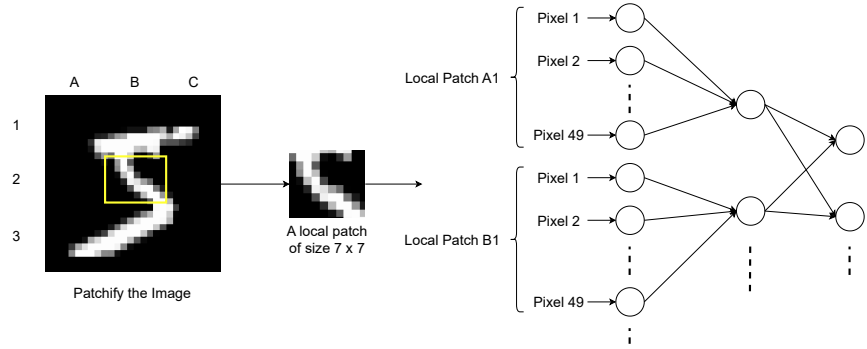
Images Have Local Structure

- What is local structure?
  - In images, nearby pixels are often related.
  - Local pixel groups form edges, corners, textures, etc.
  - These patterns (or structures) repeat across different parts of the image.
- Why this matters?
  - We don't need to look at the entire image. Only a small patch is enough to detect meaningful features.
  - Recognizing a digit when it is shifted, the local structure did not change.



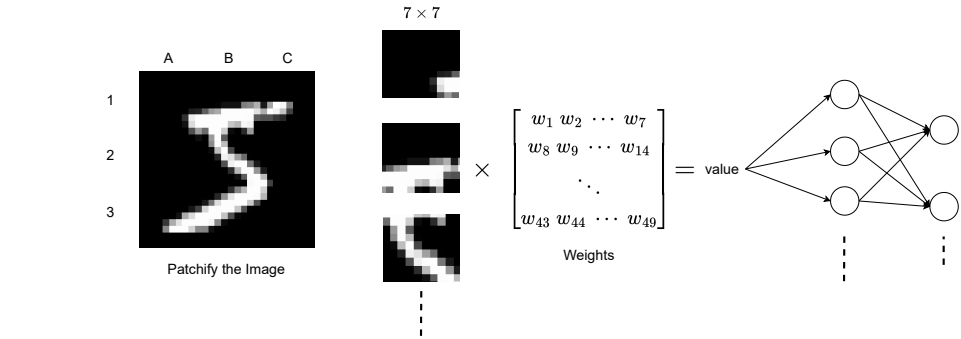
- Question: Can we **learn local patterns** and **reuse the knowledge of local patterns** from one region to another?
- Rephrasing the question, if we know how to detect local patterns (e.g., edges), can we use this knowledge to find edges in other parts of the image?

Extending MLPs: Learning Local Patterns



- **Key idea**: split the image into small patches (say 7×7), flatten them, and give the "local vectors" as inputs to the MLP.
- **Benefits**: (i) now the local structures in the image are preserved, a.k.a, local connectivity; (ii) fewer connections than a fully connected input layer, a.k.a, parameter saving.

Extending MLPs: Reusing the Knowledge of Local Patterns



- **Key idea**: instead of having a separate sets of weights for each patch, we want to have a single set of weights, and re-use them for all the patches.
- **Benefits**: (i) for e.g., if the weight matrix learned to detect some local patterns (e.g., horizontal edges), then we can re-use it to find horizontal edges everywhere in the image; (ii) more savings on parameters!

Convolutional Neural Networks

Convolutional Neural Networks

- What we discussed in the previous two slides are the two key idea behind a Convolutional Neural Network (CNN).
  - **Local connectivity**: Each neuron in a convolutional layer connects to a small patch of the input $\rightarrow$ helps model local patterns.
  - **Weight Sharing**: One set of weights (also called filter or kernel) is used across the entire image $\rightarrow$ detect the same feature anywhere in the image.
- Other benefits:
  - Huge parameter savings.
  - Baked-in translation invariance, i.e., shift the input by few pixels, the prediction of the model doesn't change.
  - Better generalization.
- CNNs are tailor-made for image data.

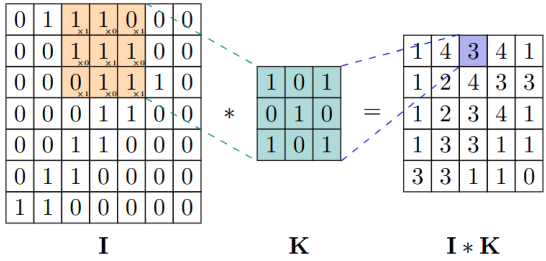
What are Filters?

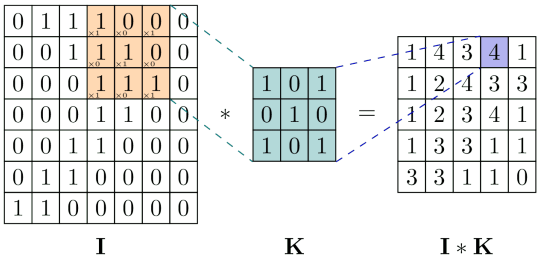
- A **filter** (or kernel) is a small matrix of numbers (weights), like 3×3 or 5×5.

$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} \qquad \begin{bmatrix} w_1 & w_2 & \cdots & w_5 \\ w_6 & w_7 & \cdots & w_{10} \\ & & \ddots & \\ w_{21} & w_{22} & \cdots & w_{25} \end{bmatrix}$$

- It works by sliding the filter across the image. At each position, it computes a dot product between the filter and the image patch. Click for animation.
- Intuitively, filters are detecting certain patterns in an image. The dot product will be high when the corresponding pattern is detected.
- It's used to detect specific local patterns in the image:
  - Edges
  - Corners
  - Textures

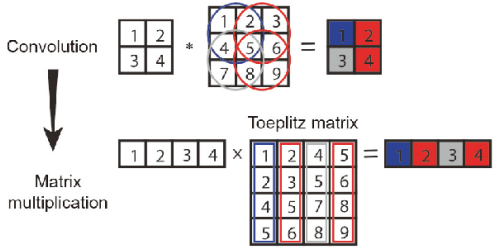
Convolution Operation



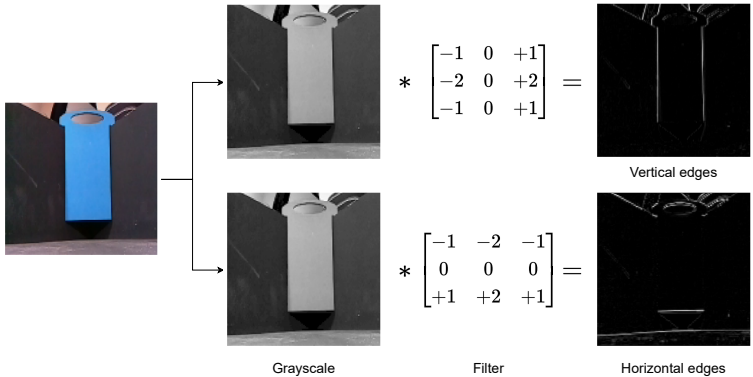


Convolution Under the Hood

- Why its called "convolutional" neural network?
  - CNNs apply filters to input data using a convolution-like operation.
  - The operation resembles the discrete convolution used in signal/image processing.
- Naive implementation of the convolution operation:
  - Inside a nested loop, slide a filter over an image, multiply overlapping values, and sum them to produce each output pixel.
  - Terrible algorithm $\rightarrow$ sequential + prohibitively slow for large images!
- Efficient implementation:



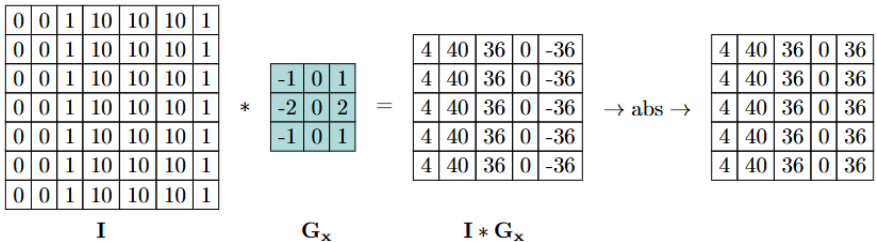
Sobel Filter



What are these filters doing?

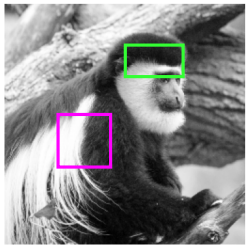
- Highlighting areas of rapid intensity change.
- In other words, its detecting edges (the places where the pixel intensity is changing rapidly).

Sobel Filter: Detecting Vertical Edges



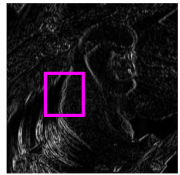
- The image $\mathbf{I}$ transitions from dark pixels on the left, i.e., pixels with low intensity (values close to zero), to bright pixels on the right, i.e., pixels with high intensity (values much higher than zero).
- When Sobel filter $\mathbf{G_x}$ is applied to a flat region the response $\mathbf{I} * \mathbf{G_x}$ is zero.
- When we apply to the transition region the response is high.
- Intuitively, Sobel filter is down-weighting the regions on the left, and up-weighting the regions on the right, so that the change of intensity is amplified.

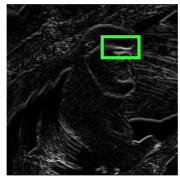
More on Sobel Filter



Grayscale

Vertical edge map





Horizontal edge map

Combined edge map

Simple Filters

Operation	Kernel ω	Image result g(x,y)
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	
Gaussian blur 5 × 5 (approximation)	$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$	

Learning the Filters

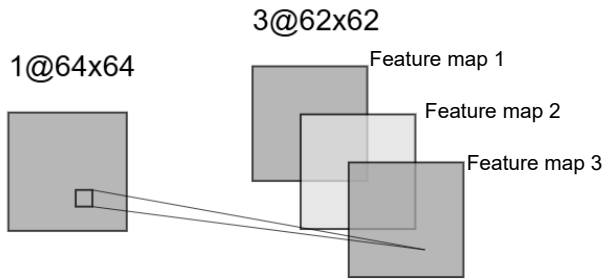
- Filters are nothing but sets of weights, the same as we have used for MLPs.
- They just look special because they are arranged in a 2D matrix, but they are not.

$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}_{3\times 3} \xrightarrow{\text{flatten}} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_9 \end{bmatrix}_{1\times 9}$$

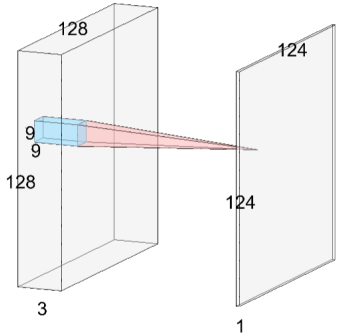
- CNNs **learn** the filters automatically through gradient descent.
- On the other hand, the Sobel filters are handcrafted filters.

Feature Maps

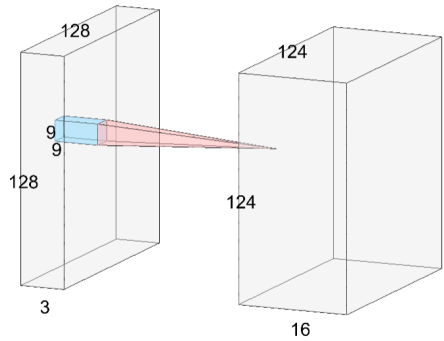
- A filter can learn only one local pattern (e.g., horizontal edge). But there can be several of these local patterns in an image.
- **Key idea**: Learn a set of filters so that each filter can learn to detect a different pattern (e.g., horizontal edge, vertical edge, edges at arbitrary angles).
- Each filter (say of size 3×3) results in one feature map (or channel). For e.g., if we have three filters, then we will get three feature maps.



Feature Maps



Filter size: $9 \times 9 \times 3 \times 1$

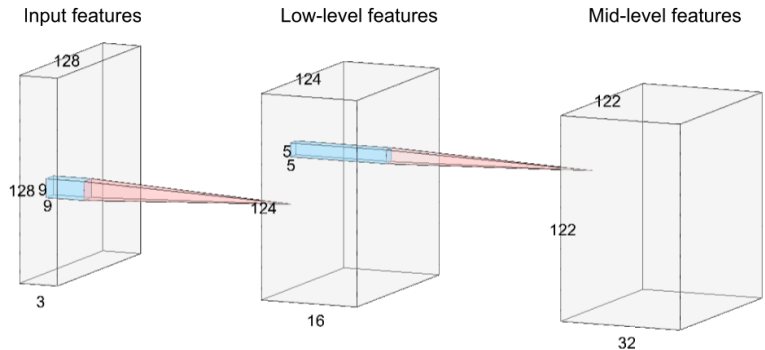


Filter size: $9 \times 9 \times 3 \times 16$

- Multiple filters (or kernels) are efficiently implemented as a single tensor with the following dimensions:

$$\text{filter dimensions} = \underbrace{9 \times 9}_{\text{kernel size}} \times \underbrace{3}_{\text{input channels}} \times \underbrace{16}_{\text{num. filters}}$$

Hierarchy of Feature Maps

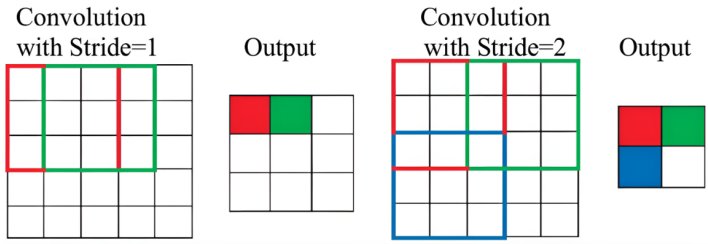


Kernel: $9 \times 9 \times 3 \times 16$ Kernel: $5 \times 5 \times 16 \times 32$

- Early filters detect simple features: edges, corners, textures.
- Deeper filters combine these features to learn more complex features: shape, parts, objects. Thus, CNNs stack multiple filters to create a hierarchy of features.

Strides in Convolution

- **Stride** denotes the number of pixels by which the filter moves after each operation.

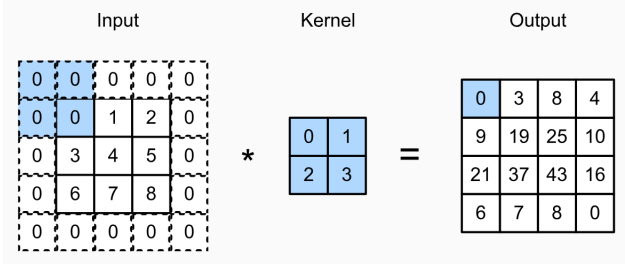


- When stride $= 1$, the kernel moves 1 pixel at a time, preserving spatial dimensions (except we lose the pixels at the borders).
- When stride $= 2$, the width and height of the output feature map is halved.
- The size of the output feature map is computed as:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 1}{\text{stride}} \right\rfloor$$

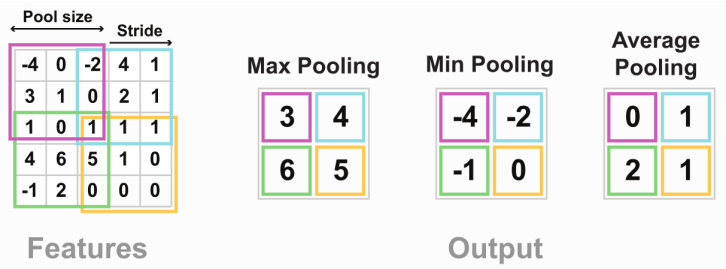
Padding

- Due to the convolution operation we lose pixels on the perimeter of the image.
- The number of pixels we lose depends on the size of the kernel. For e.g., for a 3×3 kernel, we will lose 1 pixel on each side of the image.
- This can add up as we apply many successive convolutional layers.
- **Padding** adds extra pixels to the perimeter of the input feature to preserve the dimensions of the output feature map. Eg, zero padding sets the values of the extra pixels to zero.



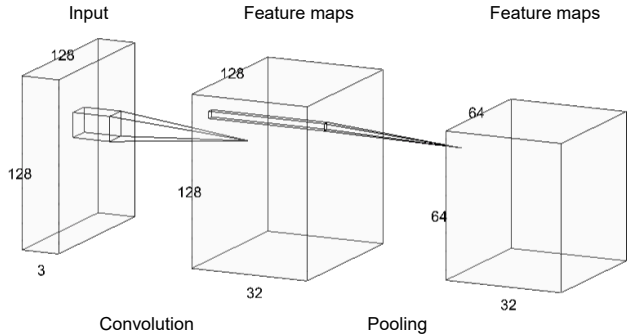
Pooling

- **Pooling** is a downsampling operation, typically applied after a convolution layer.
- There are different kinds of pooling, shown below:



- Why pooling is used?
  - Reduces the size of the feature maps $\rightarrow$ saves computation and memory.
  - Introduces translation invariance $\rightarrow$ small shifts input dont change output.

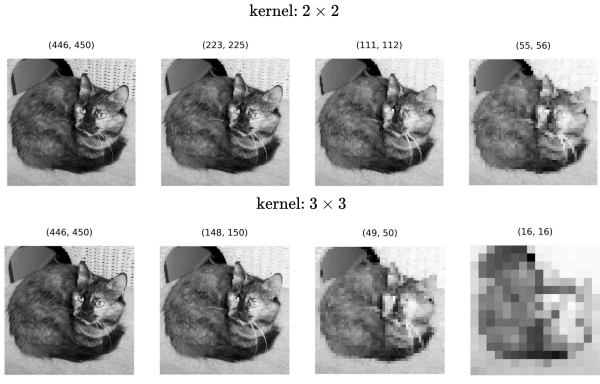
Pooling



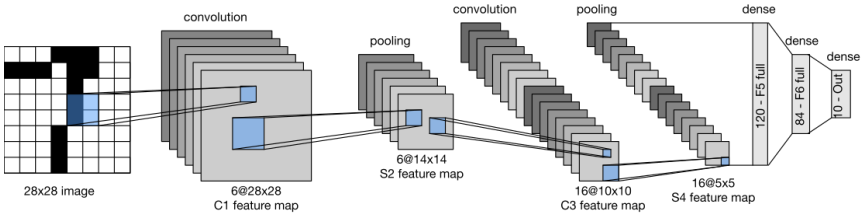
- Pooling is defined by the size of the kernel (e.g., 2×2) and the stride (e.g., 2). Note that the kernel has no weights per se.
- Pooling is applied per channel. Hence, it only reduces the spatial dimensions of the features, but does not affect the number of channels.
- Convolution with stride $= 2$ has the same effect as pooling with stride $= 2$ and size 2×2.

Pooling

- Intuitively, instead of looking at every pixel, pooling summarizes what matters in a small region (say 2×2).
- Pooling generates small sized images which retain all the essential attributes (pixels) of a input image.

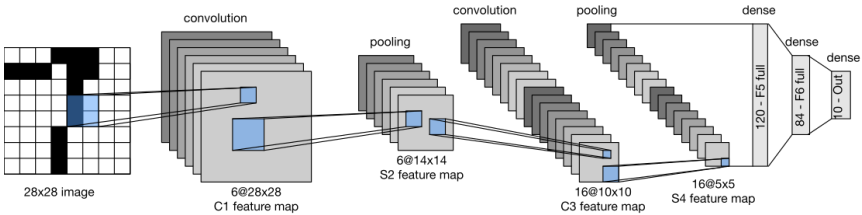


LeNet (1998): A Simple CNN For Classification



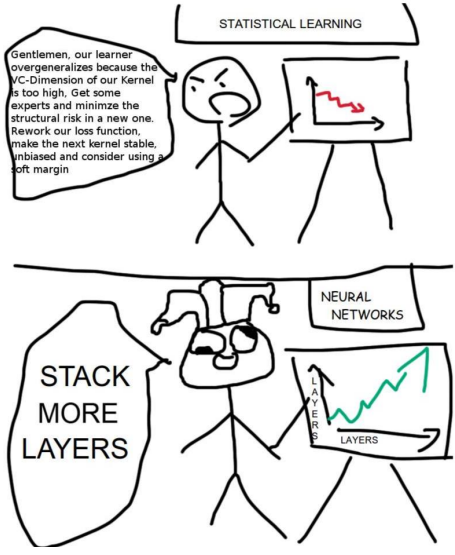
- The design and structure of different kinds of layers is called the architecture of the neural network.
- LeNet is a simple CNN that follows a repeating pattern: Conv $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$ (repeat) $\rightarrow$ Fully Connected Layer(s) $\rightarrow$ Linear Classifier $\rightarrow$ Softmax $\rightarrow$ Output
- Early layers capture low-level patterns (e.g., edges) and the later layers capture high-level patterns (e.g., shape).

LeNet (1998): A Simple CNN For Classification

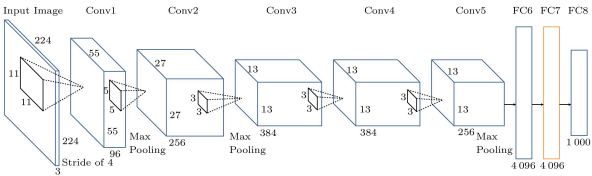


- Question: How can we go from convolutional feature maps (in layer S4) to dense features (F5)?
- We use the flatten operation that converts the 2-dimensional feature maps into a column vector, after which we can apply affine transformations in MLP.
- In pytorch we can use the function torch.flatten to perform this operation.

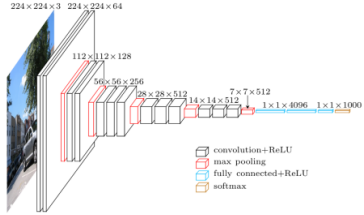
Recipe for Success: Going Deeper



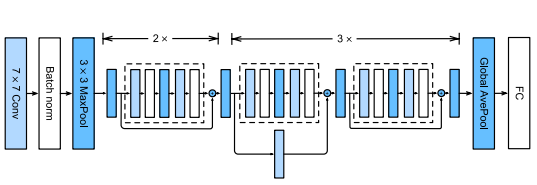
Popular CNN Architectures



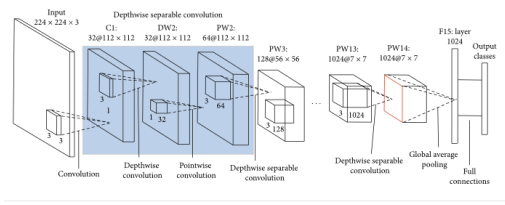
Alexnet



VGG

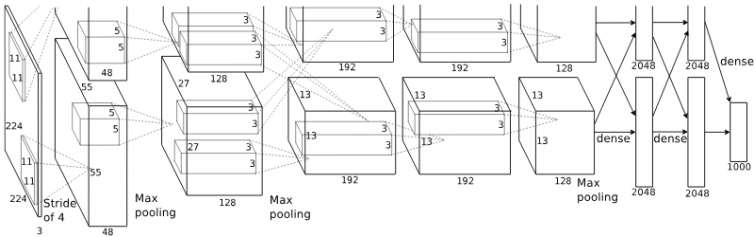


Resnet



MobileNet

Alexnet (2012)



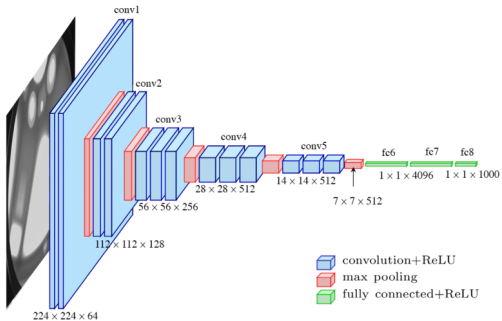
- Alexnet is a CNN that is one of the first success stories of a deeper neural network, and marked the beginning of the deep learning era.
- It broke the record for ImageNet image classification challenge (ILSVRC) in 2012, by significantly reducing the error rate ($\sim 11\%$ improvement over the runner-up).
- Recipe used in Alexnet: deeper network, trained on two GPUs, ReLU activations, dropout and data augmentation.
- The network is named after the author himself, Alex Krizhevsky.

Alexnet (2012)

- Aside from using convolution (eg, 11×11, 5×5, and 3×3 filters) and pooling operation, as used in LeNet, Alexnet[1] showed for the first time:
  - It is indeed possible to train a deeper network, going 8 layers deep.
  - GPUs can be leveraged to train DNN at a scale.
  - Speed up in training can be obtained by replacing the commonly used activation functions, ie, sigmoid/tanh, with ReLU activation function.
  - Regularization via dropout to prevent overfitting (since it has a lot of parameters).
  - Data augmentations (through randomly cropping the image or flipping) can increase the effective size of the data, adding a regularization effect.
- The recipe used in Alexnet still remain effective as of today for training much deeper DNNs.

[1] Alexnet paper

VGG (2014)



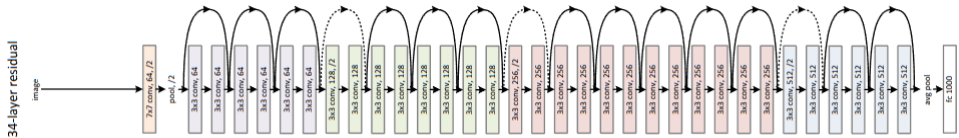
- Different from Alexnet, VGG (which stands for Visual Geometry Group in Oxford University), showed that we can go even deeper by using smaller filters of size 3×3 in all the layers.
- Outperformed Alexnet in ILSVRC 2014, by relying on the simple idea: simplicity + depth = stronger network.

VGG (2014)

- The main characteristics of VGG[2] are:
  - Instead of mixing filters of different sizes, as in Alexnet, VGG used 3×3 filters in all of the convolutional part of the backbone.
  - Stacking two 3×3 convolutions achieve the effect of using 5×5 filters, thus saving on parameters, and therefore less overfitting.
  - Because of its repetitive structure, it is customizable. VGG came in different sizes: from 11 layer deep to 19 layers deep.
- VGG became very popular and has been used in many deep learning tasks such as object detection and neural style transfer.

[2] VGG paper

ResNet (2015)



- Deep residual networks (or ResNet[3]) revolutionized depth in DNN by having networks that are numerous layers deep.
- Resnets could go very deep, even 152 layers deep, which is $8\times$ of VGG, and still not run into overfitting.
- The high-level idea in Resnet is to let layers learn only the residual (or changes), and ensure gradient flow through skip connections.

[3] Resnet paper

ResNet (2015)

- Lets denote the underlying mapping as $\mathcal{H}(\mathbf{x})$.
- Resnet casts the mapping as:

$$\mathcal{H}(\mathbf{x}) = \underbrace{\mathcal{F}(\mathbf{x})}_{\text{residual}} + \underbrace{\mathbf{x}}_{\text{skip connection}}$$

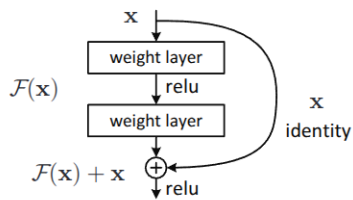
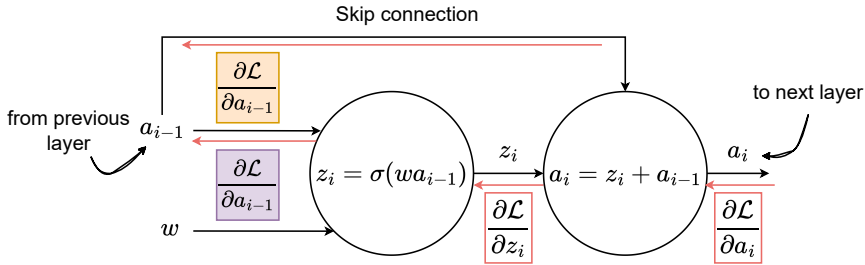


Figure 2. Residual learning: a building block.

- We simply need to learn the residual: $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$
- In case there is nothing to learn, $\mathcal{F}(\mathbf{x})$ will be driven to zero.
- The advantage of such a design is that the gradient can flow uninterrupted through the skip connections, allowing to make the network several layers deep (eg, ResNet-50, ResNet-101, and ResNet-152).

ResNet (2015)

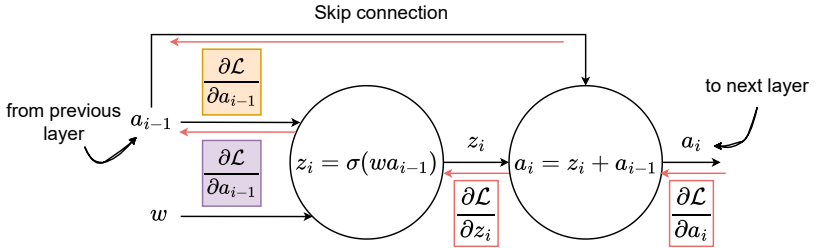


$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{\partial \mathcal{L}}{\partial a_i}\frac{\partial a_i}{\partial z_i} = \frac{\partial \mathcal{L}}{\partial a_i}\frac{\partial}{\partial z_i}(z_i + a_{i-1}) = \frac{\partial \mathcal{L}}{\partial a_i}$$

$$\frac{\partial \mathcal{L}}{\partial a_{i-1}} = \frac{\partial \mathcal{L}}{\partial z_i}\frac{\partial z_i}{\partial a_{i-1}} = \frac{\partial \mathcal{L}}{\partial a_i}\frac{\partial}{\partial a_{i-1}}\{\sigma(w a_{i-1})\} = \frac{\partial \mathcal{L}}{\partial a_i}\sigma'(w a_{i-1})\frac{\partial}{\partial a_{i-1}}(w a_{i-1})$$

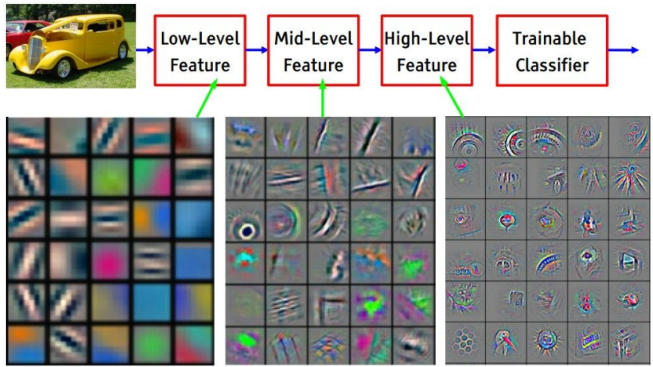
$$= \frac{\partial \mathcal{L}}{\partial a_i} \cdot \underbrace{\sigma'(w a_{i-1}) \cdot w}_{\text{can be small} = \text{vanishing gradient}}$$

ResNet (2015)



- Therefore, the upstream gradient flows uninterrupted to the previous layer through the skip connection, even if there is a vanishing gradient through the residual branch.

Visualizing What CNNs Learn



- CNNs learn filters that detect specific features at different layers:
  - Early layers → edges, corners, textures
  - Middle layers → patterns, shapes
  - Deep layers → object parts, abstract concepts

Visualizing Neural Networks

1. https://cs.stanford.edu/people/karpathy/convnetjs/index.html
2. https://adamharley.com/nn_vis/cnn/3d.html
3. https://poloclub.github.io/cnn-explainer/
4. https://playground.tensorflow.org/
5. https://blog.terencebroad.com/archive/convnetvis/vis.html

References and Links

- For additional reading materials refer to the following links
  - "Dive into Deep Learning"[4], Chapters 7 and 8.
  - "CS231n Lecture Notes"[5], Module 2.
  - "CS 230 - Deep Learning"[6].

[4] https://d2l.ai/index.html
[5] https://cs231n.github.io/
[6] https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks